Prof. Haikel Hichri, Email: hhichri@champlaincollege.qc.ca

Term Project

Milestone 1: Project design

Overview

The goal of the project is to design and implement a **RESTful Web Service application** using a clean **3-layer architecture**: **Presentation/Controller layer → Business Logic/Application/Service layer → Domain layer → Data Access/Repository layer**

Your application must use:

- RESTful API endpoints
- DTOs for communication between the API and clients
- Spring Data JPA repositories
- An in-memory **H2 database**
- Appropriate entity relationships
- Sample data loaded into the database
- Appropriate HTTP methods and status codes
- Validation where appropriate

You will publish your backend and preload it with sample data so that your REST API can be tested.

Project Choice

Choose **one** of the following topics. Your system must define at least **three core resources/entities**, including relationships between them.

1. **Hospital Appointment System: Doctor, Patient, Appointment**
2. **Event Management System: Event, Attendee, Signup**
3. **University Department System: Student, Professor, Course, Enrollment**
4. **Cinema Booking System: Customer, Movie, Screening, Booking**
5. **Social media System: User, Post, Interaction**
6. **Restaurant Reservation System: Restaurant, Customer, Reservation**
7. **Pet Clinic Management System: Pet, Owner, Veterinarian, Appointment**
8. **Car Rental System: Car, Customer, Rental**
9. **Fitness Center Management System: Member, Trainer, WorkoutPlan, Enrollment**
10. **Travel Booking System: Traveler, Destination, Booking, Trip**

Requirements

Regardless of the topic you choose, your application must:

- Define at least **three related resources/entities**.
- Implement a meaningful relationship between the resources.
- Provide CRUD operations through REST endpoints.
- Use a separate **Controller**, **Service**, and **Repository** layer.
- Use **DTOs** rather than exposing JPA entities directly through the REST API.
- Persist data using **Spring Data JPA and H2**.
- Preload the database with a reasonable set of sample data.
- Use appropriate HTTP methods such as GET, POST, PUT/PATCH, and DELETE.
- Return appropriate HTTP status codes.
- Handle invalid requests appropriately.
- Provide endpoints that allow clients to navigate or query the relationships between resources.

Additional Resources
You are free to add other entities or subdomains if they improve the realism of your application. For example, a transactional or associative entity such as **Appointment, Enrollment, Signup, Booking, Loan, Rental, or Order** can be used to represent a relationship between two resources.
The goal is not to create the largest possible system. **Focus on a well-designed, coherent REST API with clean separation of responsibilities.**

Learning Objectives

- Develop a front-end application using ReactJS (will be used in a later milestone)
- Develop RESTful web services using Spring Boot
- Integrate a React app with a RESTful web service
- Design full-stack applications using ReactJS, Spring Boot, and an SQL database
- Integrate RESTful web services with an SQL database
- Use an HTTP client to test web services (e.g., Postman, Insomnia, curl)

Part 1: Design

Conceive a multi-layer system, determine its data model, and plan the UI.

Deliverable: A short design report (PDF) including the items below.

1. Introduction

- One paragraph that explains the system's purpose, primary users, and high-level features.

2. Design Section

- Resources
    - Define at least three resources/entities.
    - Provide UML class diagrams using PUML as we have done in our activities.
    - Include DTOs: request and response DTOs for each resource.
    - Example note: For social systems, consider Post includes images or short videos.
- Relationships
    - Specify relationships (e.g., one-to-one, one-to-many, or many-to-many via join entity).
    - Provide an ERD/database schema diagram (tables, PK/FK, cardinalities).
    - Clarify cascade and fetch strategies (e.g., CascadeType.PERSIST on one-to-many, LAZY collections).
- Endpoints
    - List all REST endpoints by resource.
    - Required mappings for each resource: GET (all), GET (one by id), POST, PUT, DELETE.
    - For aggregated/relationship endpoints, provide GETs (e.g., GET /authors/{id}/books).
    - Follow REST naming conventions, plural nouns, and resource-oriented paths.
    - Include example requests/responses (JSON) and expected HTTP status codes.
- Validation
    - Identify basic validation rules (e.g., @NotBlank name, @Positive price, @Email).
    - Document expected 400 validation errors for bad input.

3. Non-Functional Considerations

- Error handling approach (NotFoundException, MethodArgumentNotValidException).
- Consistent naming & packaging conventions (feature-based or layered).
- Testing approach (at least minimal controller/service tests in a later milestone).

Submission for Part 1: PDF with the above content.